

High Level Design

~~coding~~

- ① Problem Statement
- ② Functional Requirements
- ③ Non-functional Requirements
- ④ Scale Estimation
- ⑤ Architecture Design

SDE-2 on higher positions - architectural design decisions

Any tier-1 company - have at least 1 System Design round (HLD)

Staff Engineer @ Google.] 1-2 crons per annum
10⁺ yoe in India

Interview Question for Staff Engineer @ Google → HLD round

List of strings → sort them in dictionary order

Cat
dog
apple
high level design
Scala

sort →

apple
cat
dog
high level design
Scala

Catch*

50 Petabytes of data

50,000,000 GB of data

v. large scale

Byte - 8 bits

KB - 1000 B

MB - million B

GB - billion

TB - trillion

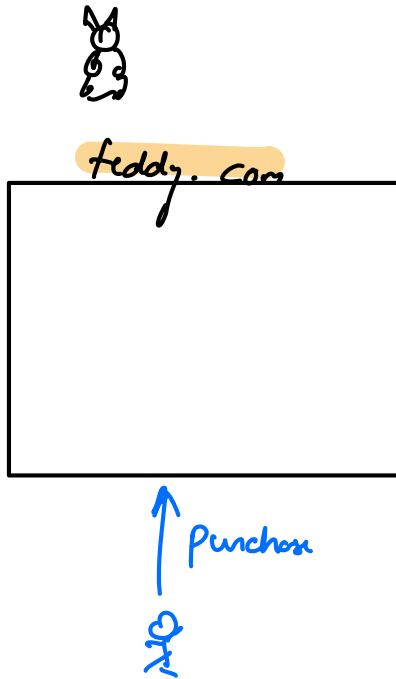
PB - quintillion

Simple problems $\xrightarrow[\text{Planet Scale}]{\text{v. large scale}}$ v. difficult

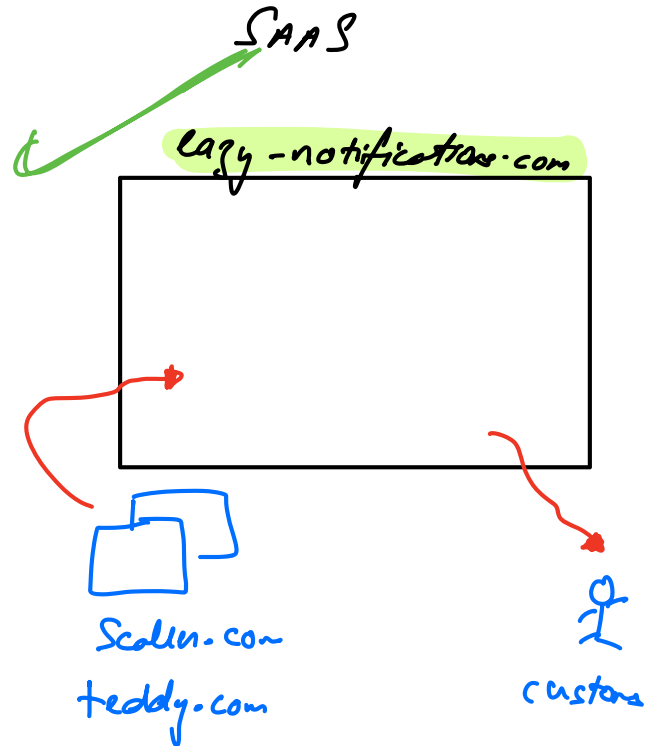
High Level Design

① Problem Statement

Notification delivery service



- Needs to send notifications to its customers
- primary function - sell product



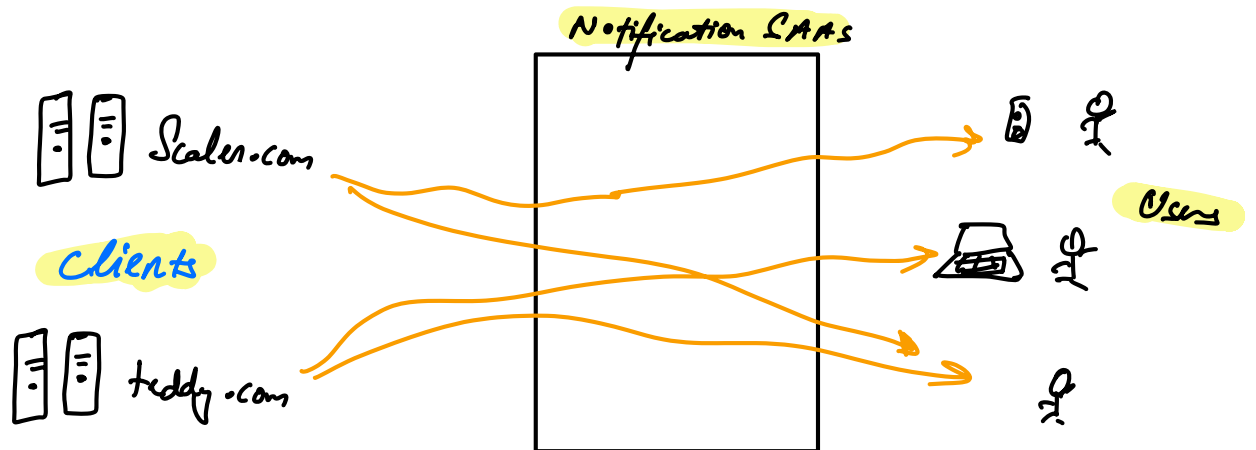
- primary function is notifications.

② Functional Requirements

MVP features

Minimal Viable Product

— don't think of fancy features



Client Requirements

① Clients should be able to send notifications to users

notify (client-id, user-id, message) → success / failure

Sender *Recipient* *Payload*

② Support multiple delivery channels

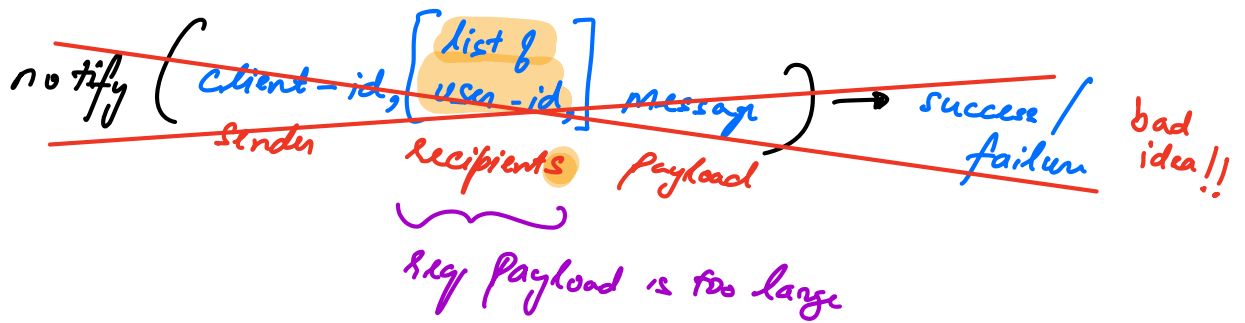
SMS / Email / in-app

Slack / WhatsApp / Pigeon

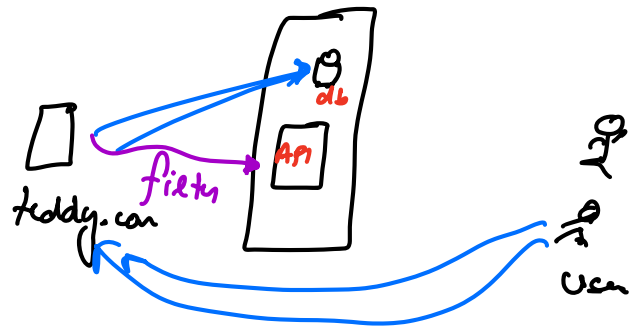
Extensible / Pluggable

We should be able to add new delivery channels without changing infrastructure

③ client should be able to broadcast messages to a group of users



- Clients already store their customer contact info in our DB
- Clients can specify a 'broadcast filter' to send bulk messages



broadcast(client-id, filter, message) → success / failure

↓

Example (user-is-subscribed-to (1234) or user-last-login > 24 hours)

User Requirements

- ① Users should be able to receive notifications
 - ↳ in real time (if they are online)
 - ↳ on re-connecting (once they come back online)
- ② users should be able to customize their notification preferences

Select channels

- ☒ Email
- ☒ SMS
- ☒ WhatsApp
- ☒ Pigeons

Notification Type

- ☒ Marketing
- ☒ Trending/News
- ☒ Financial
- ☒ OTP

Hidden Requirements

① Rate limiting

Client Limiting

free : 100 notification / day

Paid : 50,000 notification / day

enterprise : up to 50 M notification / day

User Limiting

a user should not get more than —
notification in — time

② Observability

back in 2020 Google had
~ 11 Million slaves

detecting that something is wrong

metrics like

notifications sent / hour

of errors / minute

avg CPU load

diagnose

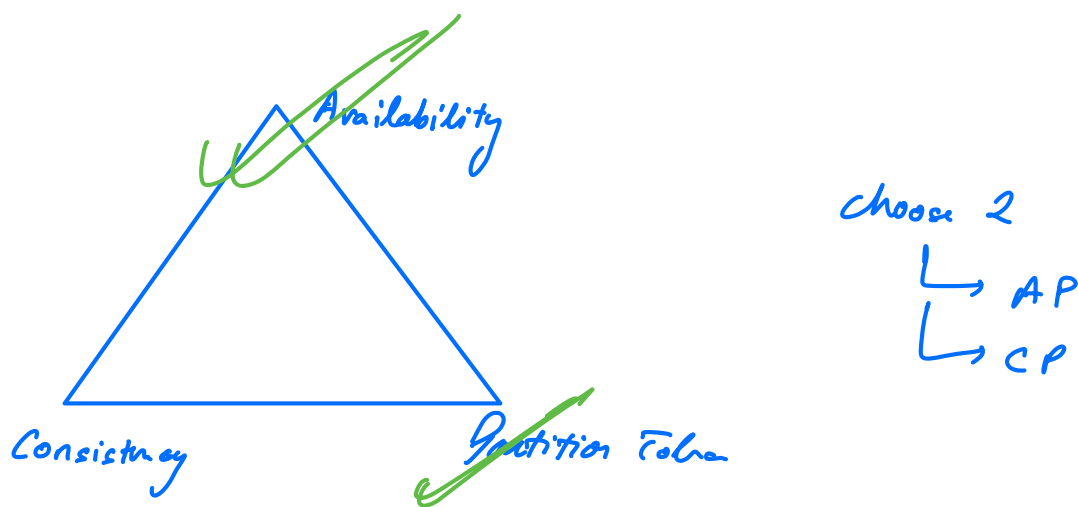
search logs

verify the fix

③ Non-functional Requirements (Design Goals)

CAP/PAL/ELC theorem

Eventual Consistency vs Immediate Consistency
for some time your data can be stale



Imagine Mala has purchased a Teddy
the teddy is out for delivery

teddy.com $\xrightarrow{\text{notify}}$ Mala

in worst case when systems are down

for some time when mala checks her phone she
gets no notification

but after some time mala gets notified

usually
mala gets notified immediately

okay

- Compliance

TRAI / FDR / ...

- Priority

OTPs should be prioritized over marketing notifications

- Exactly once delivery



- ~~• Security~~

leave this to security experts

④ Scale Estimation

- ↳ it is okay to be a little off
- ↳ it is okay to overestimate

Total # users $\approx$ 2 billion

Avg # of notifications / user / day $\approx$ 10

Ans # Notifications / second = 200,000

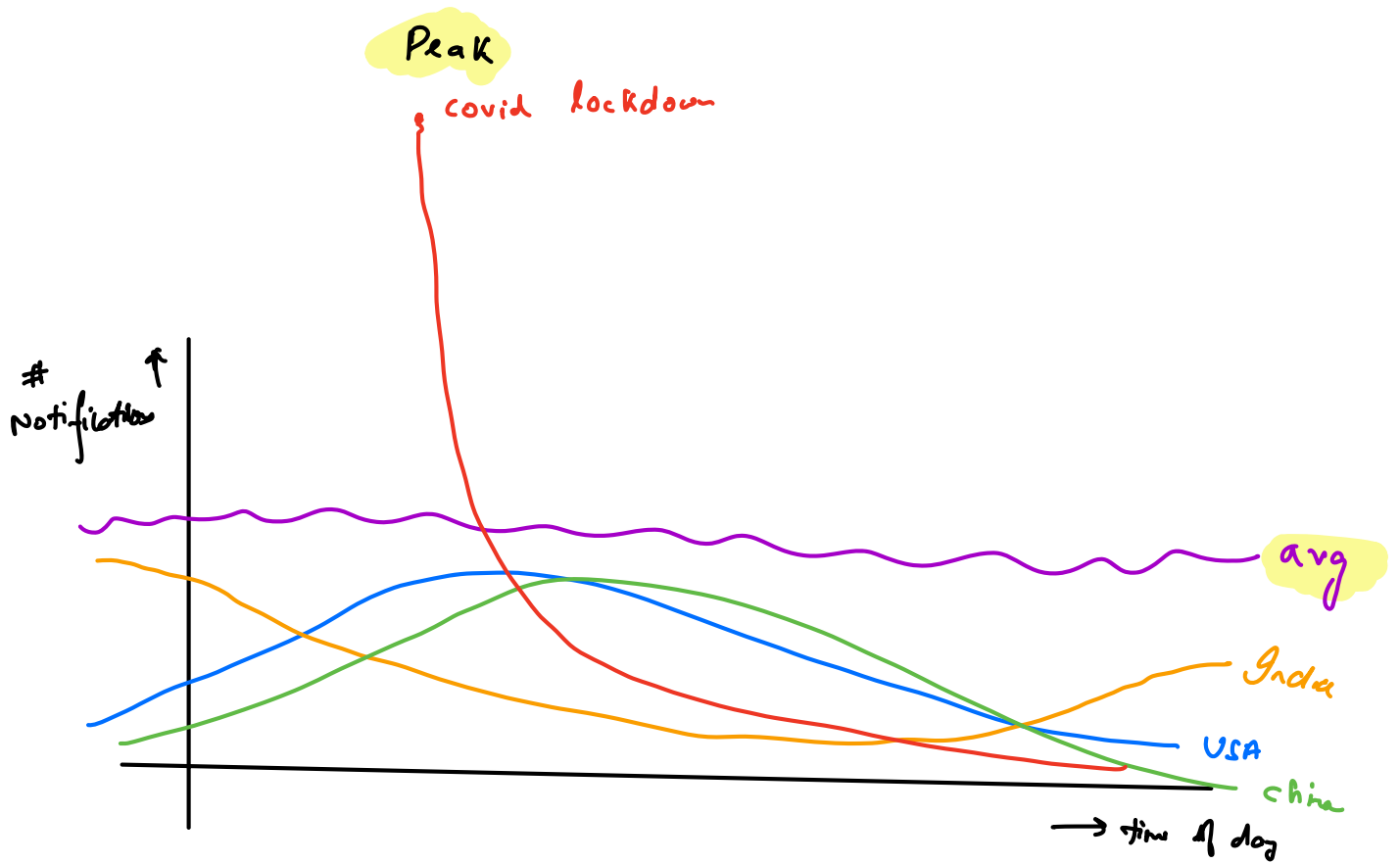
$$2 \times 10^9 \text{ users} * 10 \text{ notifications / user / day}$$

$$= 2 \times 10^{10} \text{ notifications / day}$$

$$\begin{aligned} &\rightarrow 60 * 60 * 24 \text{ seconds} \\ &= 86400 \text{ seconds} \\ &\approx 10^5 \text{ seconds} \end{aligned}$$

$$= \frac{2 \times \cancel{10^{10}}^{10^5} \text{ notifications}}{\cancel{10^5} \text{ seconds}}$$

$$\approx 200,000 \text{ notifications / sec}$$



Peak could be 50x higher than avg

$$\begin{aligned}
 \text{Peak \# notification / second} &= 200,000 \text{ avg/sec} \times 50 \\
 &= 10 \text{ million notification / sec}
 \end{aligned}$$

Jargon

- Load Balancer
- Caching
- No SQL
-

9:07 → 9:15

Scaler HLD Curriculum

- How backend systems work

DNS / vertical vs horizontal scaling / load balancing / sharding

- Load Balancing & Consistent Hashing

Healthcheck & Heartbeat / Roll over /
DNS as load balancer / Untrusted Layer /
Consistent hashing - duplicate

- Caching

Browsers / CDN / Backend

Local vs Global / Single vs Distributed

Redis

Cache Invalidation

- Write Through
- TTL
- Write back
- Write Around

Cache Eviction

- FIFO
- LRU
- LFU
- MRU

- Caching Case Studies

Scaler Code Judge / Scaler Leaderboard /
Facebook News Feed

- CAP theorem & Replication

Availability / Consistency / Partition Tolerance

Immediate — Eventual

CAP theorem / PACELC /

Master-Slave — Eventual
 Tunable Consistency — Write Everywhere
 Quorum

- SQL vs NoSQL

Pros & Cons

Sharding Key

NoSQL Types

Key-Value
 Document
 Wide Column
 Search
 File store

Graph
 Vector
 In-memory
 timeseries

- Database Internals

Orchestrator / Config Management / Reserved Servers /
 Shard Creation / Multi-master

- Database Internals

LSM Trees / Bloom Filters / Sparse Index /
 Compaction / MemTable / WAL

- Case Study — Google Typeahead
Arista / Redis / Recency / Sampling / Batching / Personalization
- Case Study — Messaging apps
Slack groups / Whatsapp / Facebook messenger
- Case Study — Kafka + ZooKeeper
event driven architecture / Pub-Sub /
topics & partitions / msg order
- Case Study — Elastic Search
Inverted Index / TF-IDF / Sharding / Tunable
Latency
- Case Study — File Storage
upload / download / chunking
- Case Study — Uber
Quad Tree / traffic patterns / nearest driver
- Case Study — Rate Limiting
DDoS / Token bucket / Leaky Bucket / Sliding Window

- Case Study - Id generation

idempotency / Unique / incremental

- Case Study - Video Streaming

Netflix / Youtube / Hotstar

Concurrency

ABS / chunking / CDNs

- Microservices

Monolith vs Microservices vs Service Oriented

Breaking a Monolith

Observability / Distributed Tracing / Logging

Thundering herd / Cascading failure / Circuit Breaker

Distributed Transactions

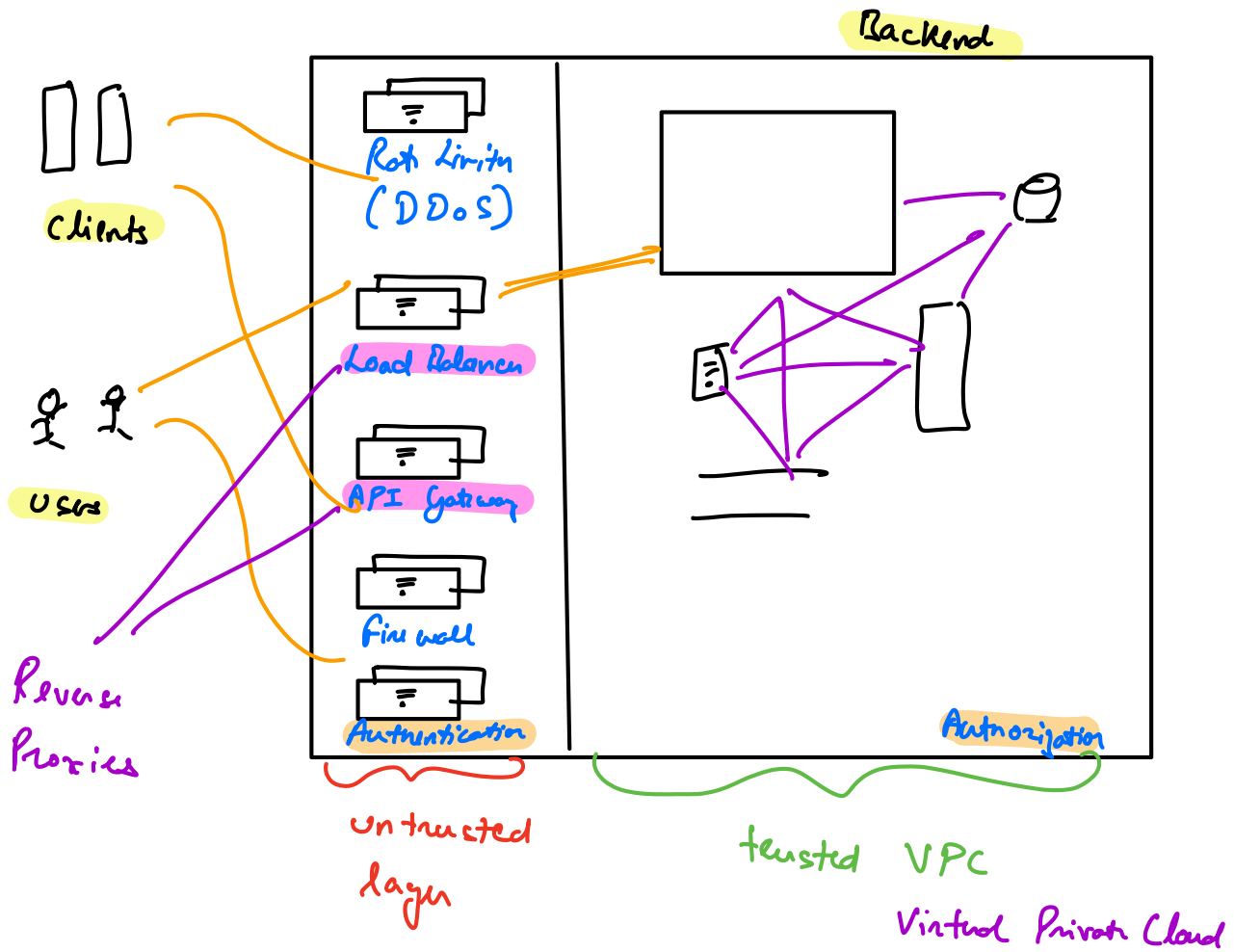
Two Phase Commit / Saga / CQRS /

Orchestration & Choreography

Event Sourcing

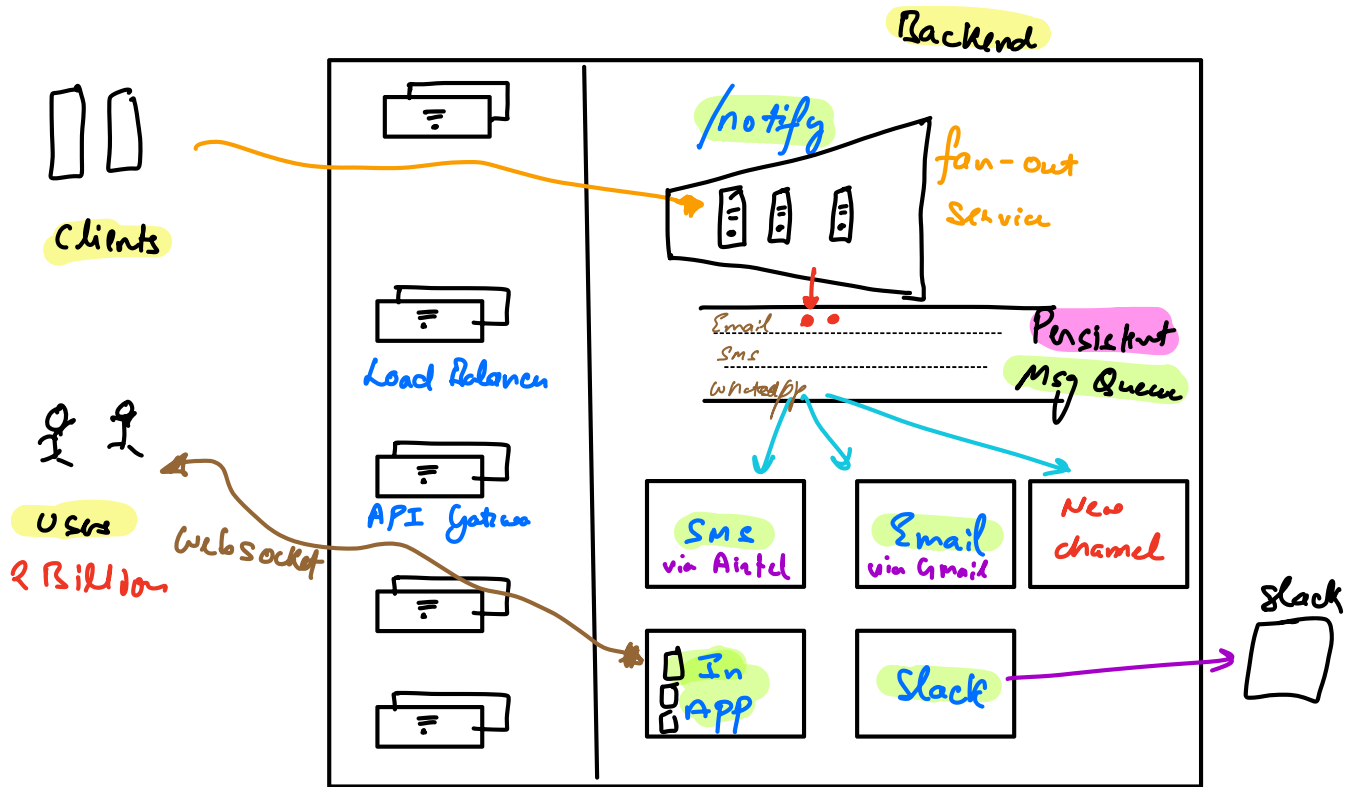
⑤ Architecture Design

Generic Design



①

Clients should be able to send notification
User should be able to receive notification



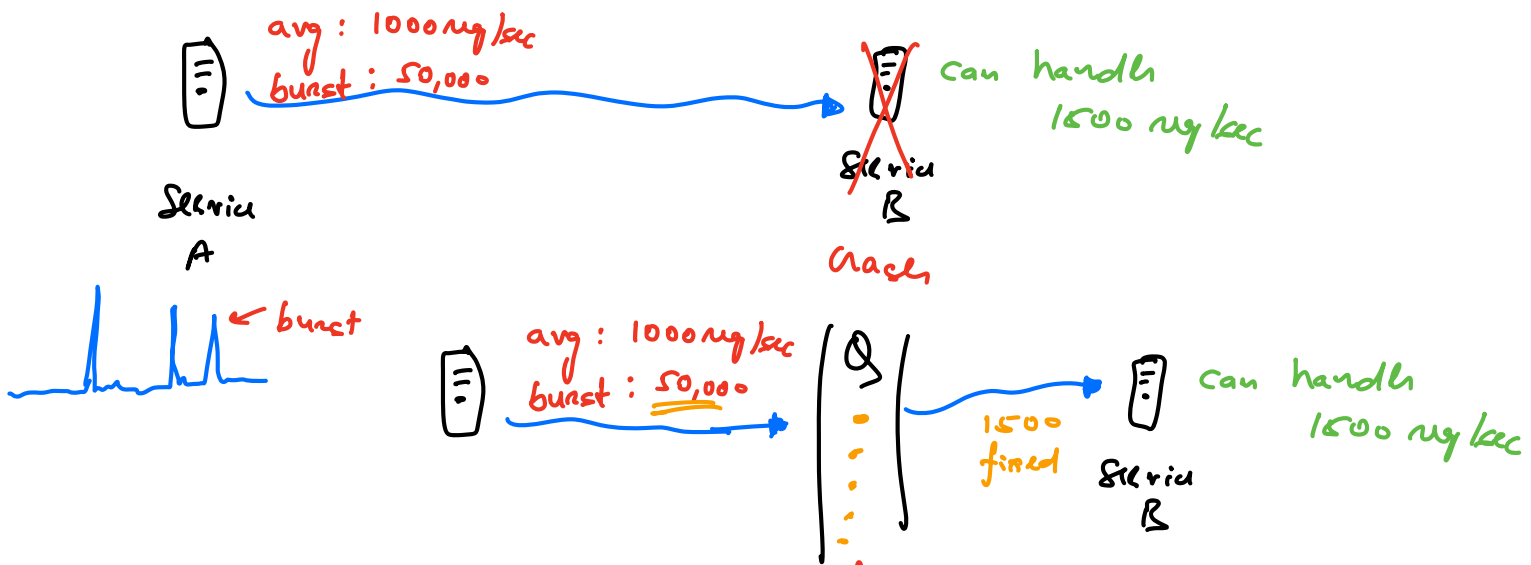
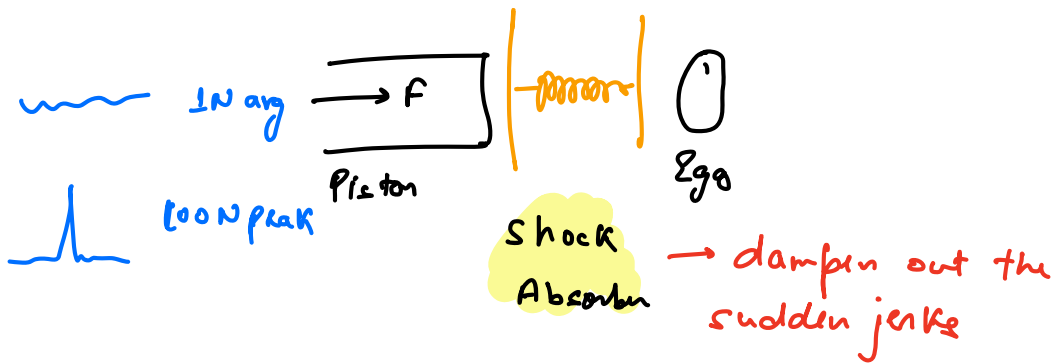
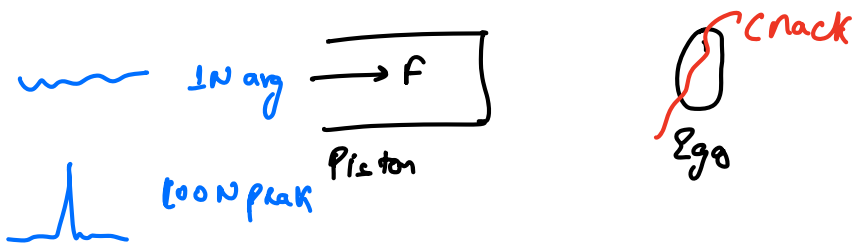
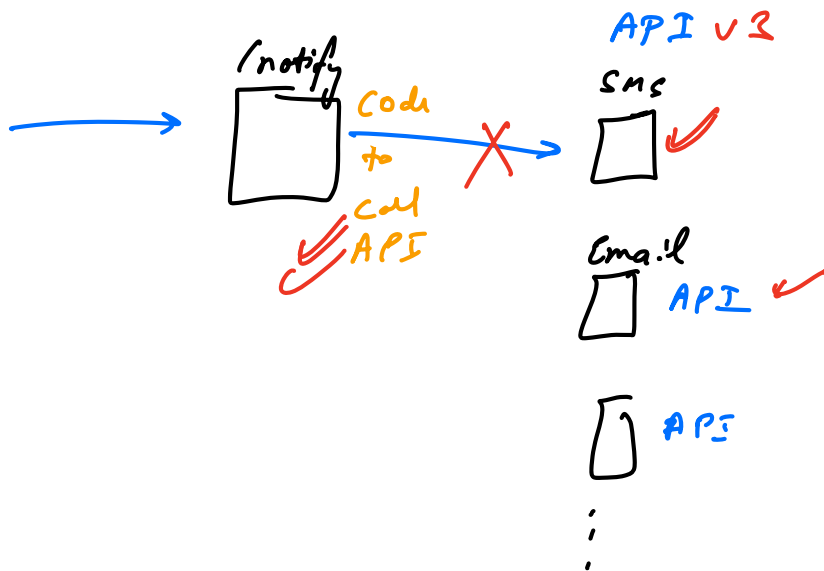
- ① user is online & connected → send notification ASAP
 - ② user can be offline → try when come back online after some time
- any notifications that occurred when they were offline
- Only for in-app

Microservices → Event Driven Architecture
Publisher Subscriber model
aka Producer Consumer.

} via Message Queue like Kafka

Pubsub = Producer Consumer

≠ Observer ≠ Event Handling



↓
Q acts as a buffer

↳ prevents notification from

↳ getting dropped

↳ provides service for crash's

notify (Client-id
User-id
message
json

msg:

type: OTP/Marketing/Finance/News
used for priority setting

channels: {

sms { msg: — emoji: —

email: { subject: ... body: ... }

} in-app: { msg: — icon: —

action-url: 'open(screen 2)'

fan-out: 1 to many



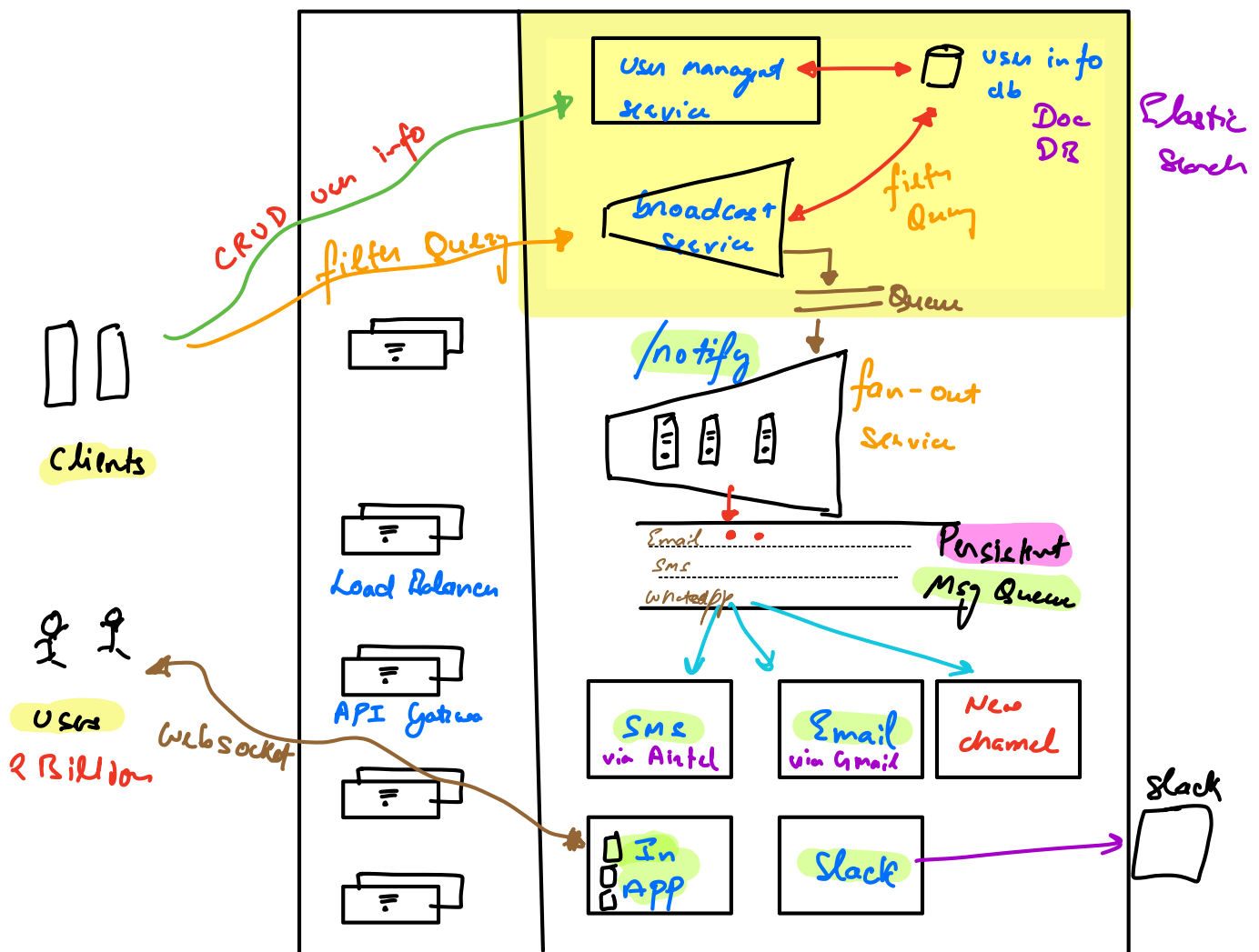
Partition the Msg Queue

- handle load
- Ensure msg order

Create various topics — one for each channel
(Kafka)

each topic can be partitioned based on the
avg load in that channel

② clients should be able to broadcast notification

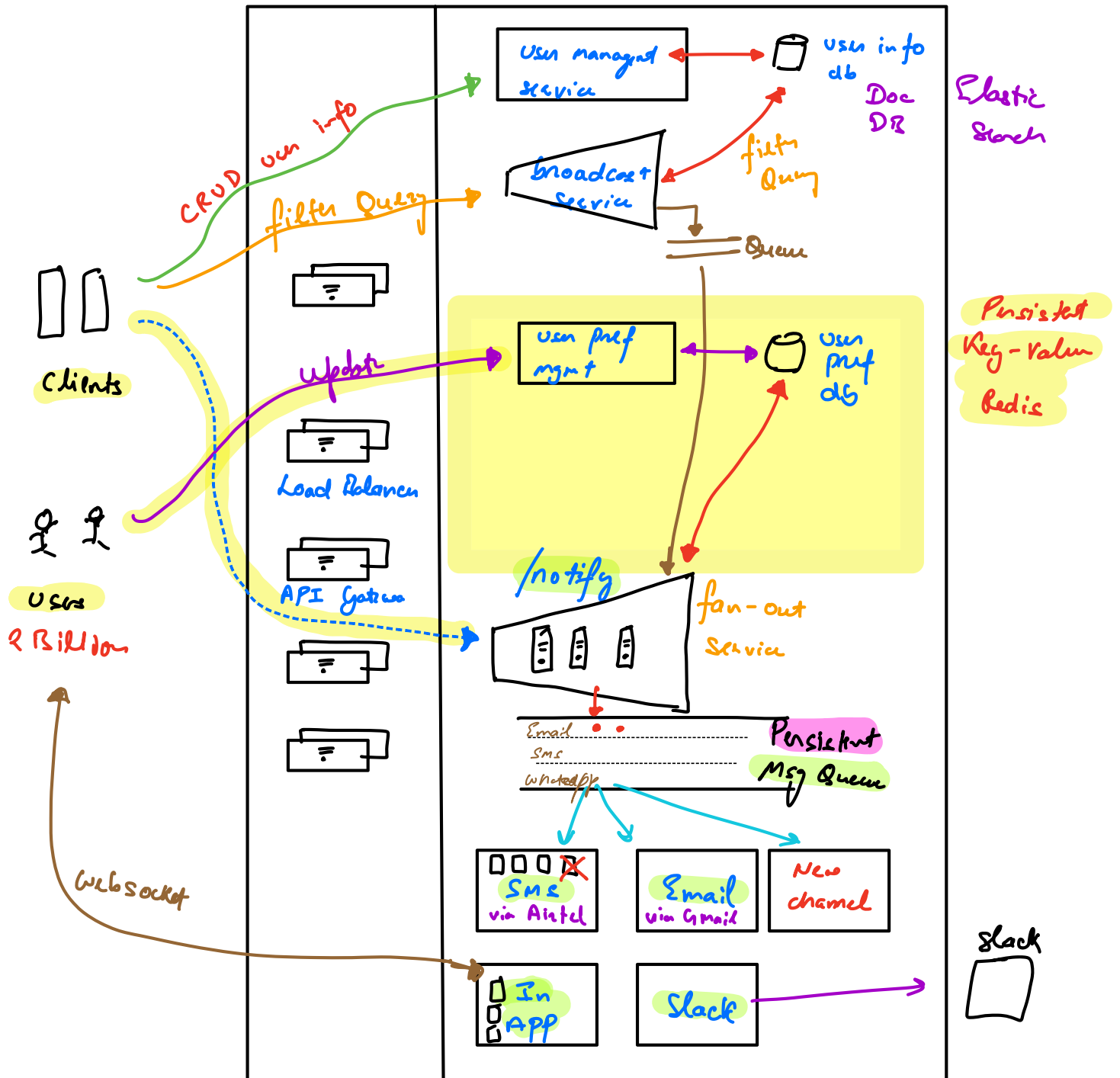


choice of DB for client - user info

	Example	Pros	Cons
SQL	Postgres My SQL SQLite	• ACID • Normalization • joins • complex queries • relations • :	• Rigid schema • don't scale horizontally (unless you shard correctly)
NoSQL — built for scale (automatic sharding, replication...)			
Document	Mongo DB Couch DB Cockroach DB Elastic Search	• Unstructured data • Full text search	• no global index • no relations • simple queries
Column	Cassandra Scylla Big Table Hbase	• fast analytics (aggregations) • pagination by time • ultra high writes	• no relations • simple queries • reading via multiple cols is slow
Key-Value	Redis Memcache Dynamo DB	• v. fast reads & writes • v. simple	• no schema • no relations • (volatile)

⑤

Users should be able to manage notification preferences



choice of db for user pref data?

- simple data — given user whether a particular channel is enabled or not

user $\rightarrow$ channel $\rightarrow$ true/false

- 10M reads/sec

Key-value (Redis + Persistence)

② Rate Limiting

$\rightarrow$ Prevent DDOS

$\rightarrow$ Client Rate Limits

$\rightarrow$ User Rate Limits

